

Lab5 Part 1: Escape Analysis and Translation-26sp

ddl 2025/5/27 23:59

1. Lab Description

In this lab, you're going to do the escape analysis and translation of the tiger compiler, you're supposed to ensure your escape analysis is correct and your translation could at least print out a complete IR tree.

Related files for this lab are:

- `src/tiger/frame/*.*` Files related to function stack frame(chapter 6)
- `src/tiger/translate/*.*` Files related to IR tree translation(chapter 7)
- `src/tiger/runtime/runtime.c` Tiger program runtime file(will be linked with code files generated by your compiler)
- `src/tiger/env/env.*` The EnvEntry classes which help the compiler store the information of variables and functions
- `src/tiger/escape/escape.*` Files related to escape analysis(chapter 6)

To finish this lab, you will only need to finish the following modules: { **x64 stack frame** } { **IR tree translation** } { **escape analysis** } you can modify any file included in the `make ziplab5`:

Code block

```
1  .
2  |—src/tiger/parse/tiger.y \
3  |—src/tiger/lex/tiger.lex \
4  |— src/tiger/lex/scanner.h \
5  |— src/tiger/semant/semant.h \
6  |— src/tiger/semant/semant.cc \
7  |— src/tiger/escape/escape.* \
8  |— src/tiger/frame/frame.h \
9  |— src/tiger/frame/temp.* \
10 |— src/tiger/frame/x64frame.* \
11 |— src/tiger/translate/translate.*
```

 **Notice:** Before you start this lab, you should carefully read chapters 6, 7, and 12 of the textbook.

Important Notes

1. Before you start to implement IR tree translation, carefully read chapters 6 and 7 of the textbook.
2. The file `runtime.c` is a C-language file containing several external functions useful to your Tiger program. These are generally reached by `externalCall` from code generated by your compiler.
3. Carefully read chapter 12, which gives some useful interface declarations you may use in the module *frame*.
4. You can only use one register(`%rsp`) as stack pointer, `%rbp` **is not allowed to be used as stack pointer**.

! This lab is quite difficult, seriously. **Please leave yourself enough time to finish this lab.** Although we don't check whether your IR tree is correct in this lab, it's very important in lab5-2, so we strongly suggest you implement it as robust and correct as possible to save your work in part 2.

Hints

1. `tr::Exp != tree::Exp`
2. For escape analysis, you'll have to implement the `Traverse` method in various classes of your AST. The purpose of this method is to walk through the tree and apply escape analysis, marking which variables escape their scope. If the variable is found and its depth is less than the current depth, it means the variable escapes (i.e., is used outside its scope). Then you should mark `escape_` as `true` in the `EscapeEntry`. Moreover, when there's no variable involved, no action is needed.
3. For the `Level` class defined in *translate.h*:
 - a. Function `Formals()` provides a way to retrieve the formal parameters of the current function's frame, transforming them into a list of `tr::Access` objects that allow access to those parameters.

- b. When a new function is entered, a **new frame** is needed. So function `NewLevel()` is used to create a new `Level` (i.e., a new function activation record), set up a new frame for the function and manage the static link between the current function and its parent function. The static link helps in accessing the variables of the parent function in the case of nested functions or closures.
4. For translating into IR tree, we offer some examples to guide you. Please refer to the [translate.cc](#) for more information.

Here we explain the implementation of translation for VarDec in detail.

Code block

```
1  tr::Exp *VarDec::Translate(env::VEnvPtr venv, env::TEnvPtr tenv,
2                               tr::Level *level, temp::Label *label,
3                               err::ErrorMsg *errmsg) const {
4      // handle the translation of variable declarations
5      /* TODO: Put your lab5 code here */
6
7      auto init_exp_ty = init->Translate(venv,tenv,level,label,errmsg);
8      type::Ty *init_ty = init_exp_ty->ty_;
9
10     // check whether the variable has an explicitly declared type
11     // and ensures that the type of the initialization expression
12     // is compatible with the declared type.
13     if (typ_) {
14         // if declared, look up the declared type in the type environment
15         type::Ty *ty = tenv->Look(typ_);
16         if (!ty) {
17             errmsg->Error(pos_, "undefined type %s", typ_->Name().data());
18         }
19         if (!ty->IsSameType(init_ty)) {
20             errmsg->Error(pos_, "type and init type mismatch");
21         }
22     } else {
23         auto actual_init_ty = init_ty->ActualTy();
24         if (typeid(*actual_init_ty) == typeid(type::NilTy)) {
25             errmsg->Error(pos_, "init should not be nil without type
26             specified");
27         }
28     }
29     // This function allocates space for the variable
30     // in the local function's stack frame.
31     // local function's stack frame is recognized according to the parameter
32     level
33     auto access = tr::Access::AllocLocal(level, escape_);
```

```

33
34     venv->Enter(var_, new env::VarEntry(access, init_ty));
35
36     // return the corresponding IR
37     return new tr::NxExp(
38         new tree::MoveStm(
39             access->access_->ToExp(
40                 new tree::TempExp(
41                     reg_manager->FramePointer()
42                 )
43             ),
44             init_exp_ty->exp_->UnEx()
45         )
46     );
47
48     /* TODO finished*/
49 }

```

2. Environment

Merge the code from lab4, and update the branch lab5-1 from gitee.

3. Local Grade Test

The lab environment contains a grading script named **grade.sh**, you can use it to evaluate your code, and that's how we grade your code, too. If you pass all the tests, the script will print a successful hint, otherwise, it will output some error messages. You can execute the script with the following commands.

Code block

```

1  ~/tiger-lab5 lab5 > make gradelab5-1
2  bash scripts/grade.sh lab5-part1
3  ===== Lab5 part-1 Test =====
4  [^_^]: Pass
5  LAB5 part 1 SCORE: 100

```

4. Submission Instructions

1. Use `make ziplab5-1` command to generate the zip file. Do not change any files other than those included in the `make ziplab5-1` command.
2. Hand in the zip file to Gradescope website.

